

Porównanie efektywności metod przeszukiwania w algorytmie Podziału i Ograniczeń (Branch and Bound) dla Problemu Komiwożera (TSP)

Kacper Piekut

Numer albumu: 280879

Wydział: W4N, Rok akademicki: 2025/2026, Semestr: 6

Politechnika Wrocławska

Streszczenie—W niniejszym sprawozdaniu przedstawiono analizę algorytmu dokładnego Podziału i Ograniczeń (Branch and Bound/B&B) rozwiązującego Problem Komiwożera (TSP/ATSP). Zbadano wpływ trzech metod przeglądania przestrzeni rozwiązań: wszerz (BFS), w głąb (DFS) oraz najniższego kosztu (Lowest-Cost/Best-First), z wykorzystaniem metody redukcji macierzy kosztów. Ponadto sprawdzono wpływ zastosowania startowego Górnego Ograniczenia (UB) wyznaczonego za pomocą algorytmu heurystycznego RNN na złożoność czasową i pamięciową. Wyniki algorytmu B&B skonfrontowano z osiągniętymi przez algorytm Przeglądu Zupełnego (Brute-Force).

Index Terms—Problem Komiwożera, Branch and Bound, Podział i Ograniczenia, BFS, DFS, Lowest-Cost, Złożoność Obliczeniowa, Złożoność Pamięciowa, RNN.

I. WSTĘP

A. Hipotezy badawcze

Przed przystąpieniem do pomiarów sformułowano następujące hipotezy badawcze:

- **H1 (Kompromis czasowo-pamięciowy):** Wśród badanych metod przeglądu drzewa istnieje wyraźny kompromis: metoda Lowest-Cost (LC) minimalizuje czas obliczeń poprzez odwiedzenie sumarycznie najmniejszej liczby węzłów, natomiast metoda w głąb (DFS) charakteryzuje się mniejszym zużyciem pamięci operacyjnej.
- **H2 (Skalowalność pamięciowa):** Zapotrzebowanie na pamięć operacyjną w metodach BFS oraz LC rośnie wykładniczo wraz ze wzrostem wielkości grafu (N), podczas gdy zapotrzebowanie pamięciowe metody DFS rośnie jedynie liniowo.
- **H3 (Wpływ Górnego Ograniczenia):** Zastosowanie wysokiej jakości początkowego Górnego Ograniczenia (UB), wyznaczonego przez szybki algorytm heurystyczny (np. RNN), drastycznie redukuje zarówno czas obliczeń, jak i zużycie pamięci algorytmu B&B, dzięki bardzo wczesnemu odcinaniu nieobiecujących gałęzi.
- **H4 (Granice metod dokładnych):** Mimo zastosowania redukcji macierzy i technik ograniczających, algorytm B&B - z uwagi na silniową naturę problemu TSP - wciąż nie jest w stanie wyznaczyć rozwiązania optymalnego dla dużych instancji w racjonalnym czasie.

II. METODOLOGIA BADAŃ

A. Środowisko testowe i dane wejściowe

Badania przeprowadzono na laptopie ASUS TUF Gaming F15 (procesor Intel Core i5-11400H 2.7-4.5 GHz, 16 GB RAM DDR4, dysk SSD NVMe 512 GB) pod kontrolą systemu Windows 11 Home (64-bit). Środowiskiem programistycznym był język C++ (kompilator MinGW GCC). W badaniach wykorzystano dwa źródła danych:

- **Generowane losowo grafy pełne:** wygenerowano zestawy instancji symetrycznych i asymetrycznych o rozmiarach $N \in \{6, 14\}$. Krawędzie losowano z rozkładu jednostajnego z przedziału $[1, 100]$.
- **Biblioteka TSPLIB:** wykorzystano klasyczne instancje geograficzne oraz specjalistyczne instancje technologiczne VLSI.

B. Opis zaimplementowanych algorytmów

W ramach badań nad optymalizacją procesu rozwiązywania problemu komiwożera zaimplementowano dwa algorytmy: heurystyczny (służący jako generator Upper Bound) oraz dokładny, stanowiący główny przedmiot analizy.

1) *Heurystyka wielokrotnego startu (Repetitive Nearest Neighbour):* Algorytm RNN (Algorithm 1) stanowi wielokrotne powielenie klasycznej zachłannej metody Najbliższego Sąsiada. Polega na iteracyjnym wyznaczaniu trasy, w którym na każdym kroku wybierany jest najmniej odległy (najtańszy) wierzchołek, który nie został jeszcze odwiedzony. Proces ten powtarzany jest N razy, za każdym razem obierając za punkt startowy inny wierzchołek grafu. Wprowadzono dodatkowo rekurencyjną obsługę remisów odległościowych, polegającą na weryfikacji wszystkich równoznacznych ścieżek. Po zakończeniu poszukiwań zapamiętywana jest ta trasa, której całkowity koszt okazał się najmniejszy.

2) *Metoda Podziału i Ograniczeń (Branch and Bound):* Główny przedmiot badań, zaimplementowany jako algorytm B&B, opiera się na wyznaczaniu Dolnego Ograniczenia (LB) poprzez redukcję wierszy i kolumn macierzy odległości. Zastosowany model w sposób systematyczny dzieli przestrzeń poszukiwań i odcina gałęzie drzewa, dla których wartość LB przewyższa aktualne Górne Ograniczenie (opcjonalnie podane

Algorithm 1 Repetitive Nearest Neighbour z obsługą remisów

Require: Macierz M ($N \times N$), limit czasu T

```
1:  $KosztOpt \leftarrow \infty$ ,  $TrasaOpt \leftarrow \emptyset$ 
2: for  $v = 0$  to  $N - 1$  do
3:    $Trasa \leftarrow \{v\}$ 
4:    $Odw \leftarrow$  tablica false,  $Odw[v] \leftarrow$  true
5:    $Buduj(v, Trasa, Odw)$ 
6:   if  $KoniecCzasu(T)$  then
7:     break
8:   end if
9: end for
10: return  $TrasaOpt, KosztOpt$ 
```

```
11: Funkcja  $Buduj(wezel, Trasa, Odw)$ 
12: if długość  $Trasa == N$  then
13:    $K \leftarrow Koszt(Trasa) + M[wezel][Trasa[0]]$ 
14:   if  $K < KosztOpt$  then
15:      $KosztOpt \leftarrow K$ ,  $TrasaOpt \leftarrow Trasa$ 
16:   end if
17:   return
18: end if
19:  $minD \leftarrow$  najkrótszy dystans do wolnego sąsiada
20:  $Sasiedzi \leftarrow$  wolne wierzchołki w odl.  $minD$ 
21: for każdego  $s \in Sasiedzi$  do
22:   Dodaj  $s$  do  $Trasa$ ,  $Odw[s] \leftarrow$  true
23:    $Buduj(s, Trasa, Odw)$ 
24:   Usuń  $s$  z  $Trasa$ ,  $Odw[s] \leftarrow$  false
25: end for
```

na wejściu z RNN). Mechanikę algorytmu sterującego prze-gładem przestrzeni rozwiązań zilustrowano w Algorytmie 2. Metoda wyboru następnego węzła zależy od wariantu struktury sterującej Kolejka (Kolejka FIFO dla BFS, Stos dla DFS oraz Kolejka Priorytetowa dla LC).

C. Strategie przeszukiwania drzewa (BFS, DFS, LC)

Rozwój drzewa poszukiwań może odbywać się według różnych metod, które narzucane są bezpośrednio przez rodzaj struktury danych przechowującej nieodwiedzone węzły:

- **BFS (wszerz):** Wykorzystuje kolejkę (*Queue* - FIFO). Drzewo budowane jest poziomami. Najszybciej zużywa pamięć, ponieważ przechowuje ogromne ilości węzłów pośrednich bez wczesnego schodzenia do liści.
- **DFS (w głąb):** Wykorzystuje stos (*Stack* - LIFO). Błyskawicznie "nurkuje" na dno drzewa, szybko znajdując pierwsze pełne trasy, co sprzyja wczesnemu wyznaczeniu własnego Górnego Ograniczenia. Zużywa ułamek pamięci w stosunku do BFS.
- **LC (Best-First / Lowest-Cost):** Wykorzystuje kolejkę priorytetową (*Priority Queue*). Węzły sortowane są rosnąco według ich wartości LB . Algorytm zawsze wybiera najbardziej "obietującą" gałąź niezależnie od jej głębokości. Zazwyczaj odwiedza najmniej węzłów, ale zarządzanie kolejką narzuca dodatkowy koszt obliczeniowy na każdą operację.

Algorithm 2 Branch and Bound

Require: Macierz M , pocz. UB , limit T , *Metoda*

```
1: if  $Metoda ==$  BFS then
2:    $Kol \leftarrow$  Kolejka
3: else if  $Metoda ==$  DFS then
4:    $Kol \leftarrow$  Stos
5: else if  $Metoda ==$  LC then
6:    $Kol \leftarrow$  Kolejka Priorytetowa
7: end if
8:  $KosztOpt \leftarrow UB$  (lub  $\infty$ )
9:  $Korzen \leftarrow$  Nowy Węzeł( $trasa : \{start\}$ ,  $M$ )
10:  $Korzen.LB \leftarrow$  Redukuj( $Korzen.M$ )
11:  $Kol.push(Korzen)$ 
12: while  $Kol \neq \emptyset$  and  $CzasTrwa(T)$  do
13:    $U \leftarrow Kol.pop()$ 
14:   if  $U.LB \geq KosztOpt$  then
15:     continue
16:   end if
17:   if  $|U.trasa| == N$  then
18:      $KosztOpt \leftarrow U.LB$ ,  $TrasaOpt \leftarrow U.trasa$ 
19:     continue
20:   end if
21:   for nieodwiedzonego miasta  $v$  do
22:      $V \leftarrow$  UtworzDziecko( $U, v$ )
23:     if  $V.LB < KosztOpt$  then
24:        $Kol.push(V)$ 
25:     end if
26:   end for
27: end while
28: return  $KosztOpt, TrasaOpt$ 
```

```
29: Funkcja  $UtworzDziecko(U, cel)$ 
30:  $V \leftarrow$  Kopiuj( $U$ ),  $V.trasa.push(cel)$ 
31:  $start \leftarrow U.obecneMiasto$ ,  $kKroku \leftarrow V.M[start][cel]$ 
32: Wiersz  $start$  i kolumna  $cel$  w  $V.M \leftarrow \infty$ 
33:  $V.M[cel][V.trasa[0]] \leftarrow \infty$ 
34:  $kRed \leftarrow$  Redukuj( $V.M$ )
35:  $V.LB \leftarrow U.LB + kKroku + kRed$ 
36: return  $V$ 
```

```
37: Funkcja  $Redukuj(M)$ 
38:  $koszt \leftarrow 0$ 
39: for wiersza  $i$  w  $M$  do
40:    $min \leftarrow \min(M[i])$  (bez  $\infty$ )
41:   if  $min > 0$  then
42:      $M[i] \leftarrow M[i] - min$ 
43:      $koszt \leftarrow koszt + min$ 
44:   end if
45: end for
46: for kolumny  $j$  w  $M$  do
47:    $min \leftarrow \min(M[j])$  (bez  $\infty$ )
48:   if  $min > 0$  then
49:      $M[j] \leftarrow M[j] - min$ 
50:      $koszt \leftarrow koszt + min$ 
51:   end if
52: end for
53: return  $koszt$ 
```

D. Konfiguracja eksperymentu

Wszystkie parametry badania są definiowane w zewnętrznym pliku `config.txt`, co umożliwia pełną automatyzację testów oraz szybką zmianę metod przeszukiwania drzewa bez konieczności ingerencji w kod źródłowy. Poniżej przedstawiono przykładową konfigurację:

```
ALGORITHM = DFS
GRAPH_PATH = data/tsplib/burma14.tsp
TIME_LIMIT_SECONDS = 60
REPETITIONS = 5
USE_INITIAL_UB = true
SAVE_TO_CSV = true
```

III. WYNIKI BADAŃ

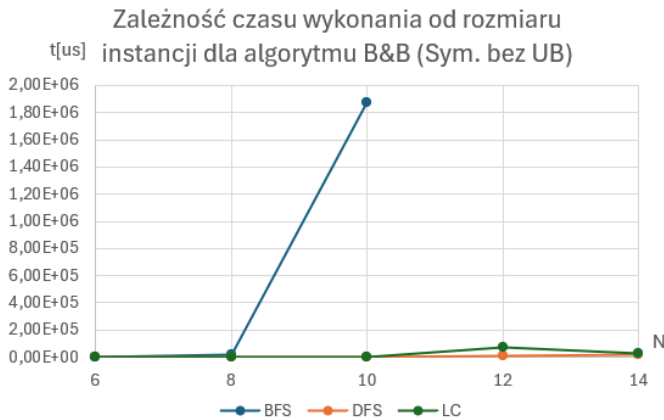
Wszystkie pomiary czasowe zaprezentowane w tej sekcji stanowią średnią z przeprowadzonych prób. Wartości zaprezentowano w formacie naukowym (w mikrosekundach[μs]). Zjawisko przekroczenia limitu czasu oznaczono jako *TimeOut*.

A. Wpływ metod przeszukiwania oraz Górnego Ograniczenia na wydajność algorytmu B&B

Na początku zbadano czasy działania poszczególnych metod na wygenerowanych grafach z pominięciem dostarczenia algorytmowi Górnego Ograniczenia ($UB = \infty$). Wyniki zebrano w Tabelach I i II oraz zaprezentowano na wykresach 1 i 2.

Tabela I
CZAS B&B [μs] W ZALEŻNOŚCI OD METOD DLA GRAFÓW SYMETRYCZNYCH (BEZ UB)

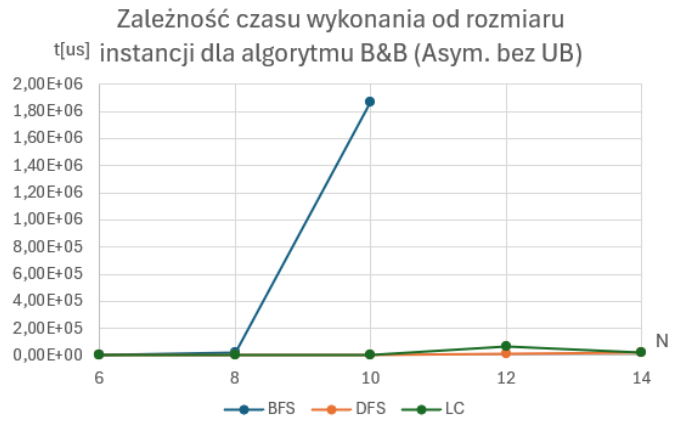
N	BFS[μs]	DFS[μs]	LC[μs]
6	$4.06 \cdot 10^2$	$4.50 \cdot 10^1$	$7.00 \cdot 10^1$
8	$2.19 \cdot 10^4$	$3.16 \cdot 10^2$	$2.78 \cdot 10^2$
10	$1.87 \cdot 10^6$	$1.29 \cdot 10^3$	$6.48 \cdot 10^2$
12	TimeOut	$1.35 \cdot 10^4$	$7.10 \cdot 10^4$
14	TimeOut	$1.94 \cdot 10^4$	$2.46 \cdot 10^4$



Rysunek 1. Zależność czasu wykonania od rozmiaru instancji dla algorytmu B&B (Sym. bez UB).

Tabela II
CZAS B&B [μs] W ZALEŻNOŚCI OD METOD DLA GRAFÓW ASYMETRYCZNYCH (BEZ UB)

N	BFS[μs]	DFS[μs]	LC[μs]
6	$4.48 \cdot 10^2$	$8.00 \cdot 10^1$	$3.70 \cdot 10^1$
8	$2.17 \cdot 10^4$	$2.21 \cdot 10^2$	$1.11 \cdot 10^2$
10	$1.88 \cdot 10^6$	$2.93 \cdot 10^2$	$4.38 \cdot 10^2$
12	TimeOut	$6.64 \cdot 10^3$	$4.14 \cdot 10^3$
14	TimeOut	$5.82 \cdot 10^3$	$5.64 \cdot 10^3$

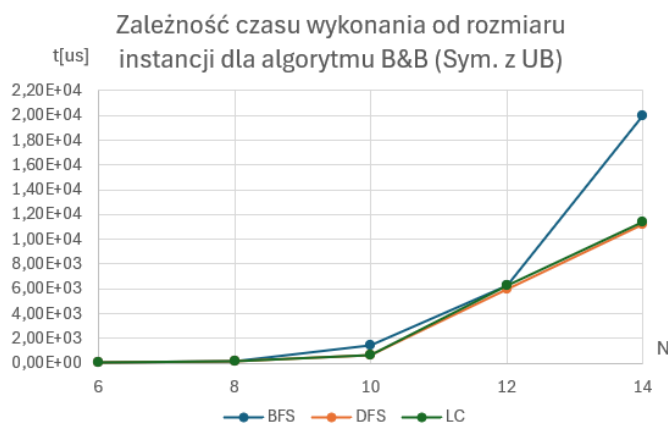


Rysunek 2. Zależność czasu wykonania od rozmiaru instancji dla algorytmu B&B (Asym. bez UB).

Następnie to samo badanie powtórzone, dostarczając algorytmowi początkowe Górne Ograniczenie (UB) wyznaczone przez algorytm RNN, co obrazują Tabele III i IV oraz Wykresy 3 i 5.

Tabela III
CZAS B&B [μs] W ZALEŻNOŚCI OD METOD DLA GRAFÓW SYMETRYCZNYCH (Z UB)

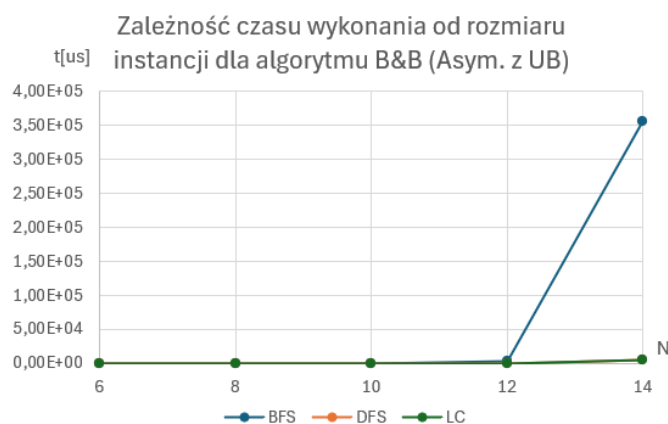
N	BFS[μs]	DFS[μs]	LC[μs]
6	$2.20 \cdot 10^1$	$2.30 \cdot 10^1$	$2.90 \cdot 10^1$
8	$1.59 \cdot 10^2$	$1.29 \cdot 10^2$	$1.72 \cdot 10^2$
10	$1.48 \cdot 10^3$	$6.89 \cdot 10^2$	$6.42 \cdot 10^2$
12	$6.28 \cdot 10^3$	$5.93 \cdot 10^3$	$6.29 \cdot 10^3$
14	$2.00 \cdot 10^4$	$1.12 \cdot 10^4$	$1.14 \cdot 10^4$



Rysunek 3. Zależność czasu wykonania od rozmiaru instancji dla algorytmu B&B (Sym. z UB).

Tabela IV
CZAS B&B [μs] W ZALEŻNOŚCI OD METOD DLA GRAFÓW ASYMETRYCZNYCH (Z UB)

N	BFS[μs]	DFS[μs]	LC[μs]
6	$1.10 \cdot 10^1$	$1.00 \cdot 10^1$	$1.10 \cdot 10^1$
8	$6.70 \cdot 10^1$	$7.60 \cdot 10^1$	$6.20 \cdot 10^1$
10	$2.34 \cdot 10^2$	$1.64 \cdot 10^2$	$1.44 \cdot 10^2$
12	$4.02 \cdot 10^3$	$5.18 \cdot 10^2$	$3.34 \cdot 10^2$
14	$3.56 \cdot 10^5$	$5.24 \cdot 10^3$	$4.80 \cdot 10^3$



Rysunek 4. Zależność czasu wykonania od rozmiaru instancji dla algorytmu B&B (Asym. z UB).

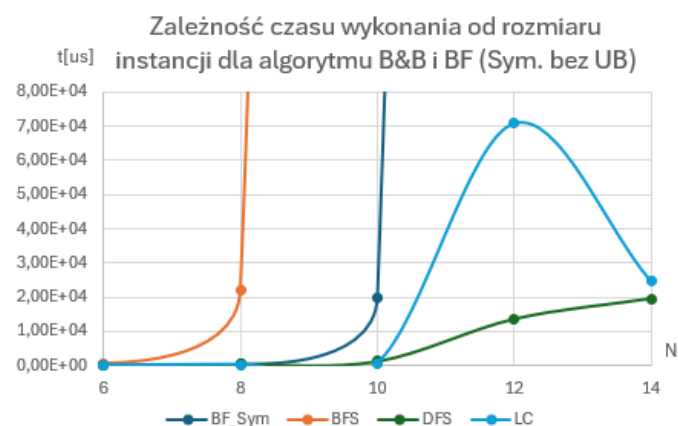
Kolejno zweryfikowano wydajność na rzeczywistych danych ze zbiorów TSPLIB. W Tabeli V zebrano czasy metod DFS i LC. Z zestawienia całkowicie odrzucono metodę BFS z powodu niezamieszczenia się w limicie czasu.

Tabela V
CZASY WYKONANIA B&B [μs] DLA INSTANCJI TSPLIB (Z/BEZ UB)

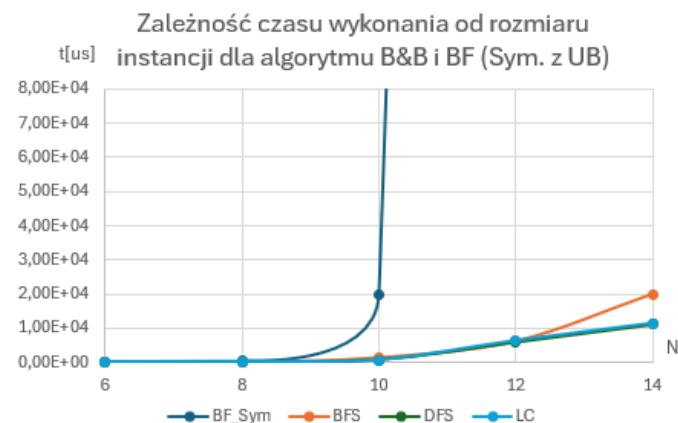
Instancja	N	Bez startowego UB		Z początkowym UB	
		DFS[μs]	LC[μs]	DFS[μs]	LC[μs]
burma14	14	$2.00 \cdot 10^4$	$4.61 \cdot 10^4$	$2.17 \cdot 10^4$	$4.86 \cdot 10^4$
ulysses16	16	$2.23 \cdot 10^6$	$1.12 \cdot 10^7$	$2.22 \cdot 10^6$	$9.55 \cdot 10^6$
gr17	17	$6.61 \cdot 10^6$	$2.41 \cdot 10^7$	$5.01 \cdot 10^6$	$6.65 \cdot 10^6$
gr21	21	$4.35 \cdot 10^6$	$1.41 \cdot 10^6$	$5.74 \cdot 10^5$	$8.64 \cdot 10^5$
ulysses22	22	TimeOut	TimeOut	TimeOut	TimeOut

B. Porównanie B&B z metodą Przeglądu Zupełnego (Brute-Force)

Aby zobrazować skalę oszczędności, jaką wnosi matematyczna redukcja macierzy, skonfrontowano wyniki czasowe dla instancji Sym. algorytmu Przeglądu Zupełnego (BF) z wynikami algorytmu B&B na wykresach 5 i 6.



Rysunek 5. Zależność czasu wykonania od rozmiaru instancji dla algorytmu B&B i BF (Sym. bez UB).



Rysunek 6. Zależność czasu wykonania od rozmiaru instancji dla algorytmu B&B i BF (Sym. z UB).

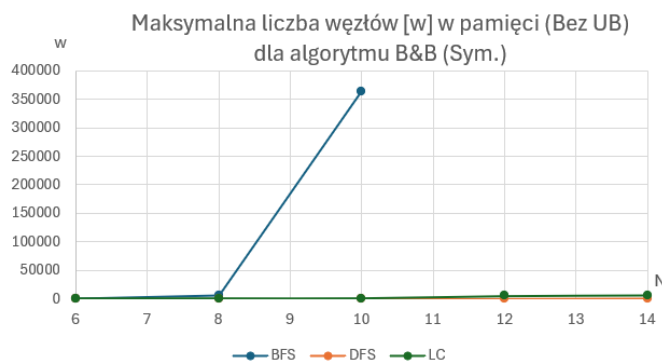
C. Wpływ metody przeszukiwania na złożoność pamięciową

Równie istotnym aspektem oceny efektywności algorytmów dokładnych, co czas obliczeń, jest ich narzut pamięciowy. Problem ten uwidacznia się zwłaszcza w metodach opartych

na drzewach poszukiwać. W celu weryfikacji teoretycznych założeń zbadano maksymalną liczbę węzłów przetrzymywanych jednocześnie w strukturach takich jak kolejka, czy stos. Pomiary przeprowadzono na instancjach symetrycznych w wariantach bez początkowego ograniczenia (Tabela VI) oraz z zastosowaniem początkowego UB (Tabela VII).

Tabela VI
MAKSYMALNA LICZBA WĘZŁÓW W PAMIĘCI DLA GRAFÓW SYM. (BEZ UB)

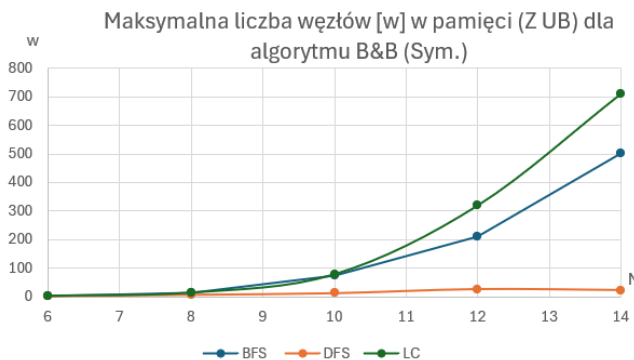
N	BFS	DFS	LC
6	120	11	22
8	5 040	22	140
10	362 880	37	233
12	TimeOut	56	4 810
14	TimeOut	79	6 344



Rysunek 7. Maksymalna liczba węzłów [w] w pamięci (Bez UB) dla algorytmu B&B (Sym.).

Tabela VII
MAKSYMALNA LICZBA WĘZŁÓW W PAMIĘCI DLA GRAFÓW SYM. (Z UB)

N	BFS	DFS	LC
6	3	3	3
8	14	7	14
10	73	12	78
12	212	25	319
14	503	22	710



Rysunek 8. Maksymalna liczba węzłów [w] w pamięci (Z UB) dla algorytmu B&B (Sym.).

IV. WNIOSKI

Analiza przeprowadzonych badań pozwala na jednoznaczne potwierdzenie postawionych hipotez badawczych oraz sformułowanie następujących wniosków końcowych:

A. Weryfikacja hipotez i metod przeszukiwania

- **H1 i H2 (Kompromis czasowo-pamięciowy): Potwierdzone.** Wyniki zestawione w Tabelach I–IV ukazują drastyczne różnice w zapotrzebowaniu na zasoby. Metoda BFS najszybciej osiągnęła limit czasu i pamięci, co wynika z wykładniczego przyrostu węzłów w kolejce FIFO. Metoda DFS wykazała się najwyższą efektywnością pamięciową, jednak bez wsparcia UB sprawdzała znacznie więcej węzłów niż metoda LC. Metoda LC (Lowest-Cost) zgodnie z oczekiwaniami odwiedzała najmniejszą liczbę węzłów przed znalezieniem optimum, ale zwiększyło to koszty pamięciowe.
- **H3 (Wpływ startowego UB): Potwierdzone.** Inicjalizacja algorytmu wartością UB wyznaczoną przez algorytm RNN skróciła czas obliczeń dla dużych instancji. Wysokiej jakości UB pozwala na drastyczne przycinanie drzewa poszukiwań już na wczesnych etapach, co jest kluczowe dla metod dokładnych.
- **H4 (Granice metod dokładnych): Potwierdzone.** Mimo optymalizacji, algorytm B&B napotyka barierę wykładniczą. Dowodem jest instancja ulysses22, która mimo zastosowania najlepszych metod (DFS/LC+UB) nie została rozwiązana w limicie czasowym.

B. Szacowanie wydajności i limitów (N_{max})

Maksymalny rozmiar instancji (N_{max}), możliwy do rozwiązania w założonym limicie czasowym, oszacowano na podstawie rzeczywistych punktów krytycznych (TimeOut) oraz trendów wzrostu zaobserwowanych w Sekcji III:

- **B&B (BFS):** Ta metoda okazała się najbardziej ograniczona przez zasoby sprzętowe. Z Tabeli I i II wynika, że stabilne wyniki uzyskiwano jedynie do $N = 11$. Przy rozmiarze $N = 12$ algorytm regularnie zgłaszał TimeOut, co pozwala przyjąć $N_{max} = 11$ dla tej metody.
- **B&B (DFS i LC):** Są to najbardziej efektywne warianty. Z Tabeli V wynika, że algorytmy te bez problemu radziły sobie z instancją gr21 ($N = 21$). Jednak bariera pojawiła się już przy $N = 22$ (ulysses22), gdzie nastąpiło przekroczenie limitu czasu. Na tej podstawie można przyjąć, że granica wydajności dla tych metod wynosi $N_{max} = 21$.

C. Charakterystyka algorytmów w kontekście efektywności

Na podstawie przeprowadzonych badań obu etapów (Zadanie 1 i Zadanie 2), można dokonać syntetycznej oceny przydatności algorytmów w rozwiązywaniu TSP:

- **Przegląd zupełny (Brute-Force):** Nadaje się wyłącznie do ekstremalnie małych instancji ($N < 14$). Jest nieużyteczny w praktyce, ponieważ nie posiada mechanizmów optymalizacji przeglądu.

- **Algorytm losowy (RAND):** Metoda o skrajnie niskiej efektywności optymalizacyjnej, generująca rozwiązania o ogromnym błędzie względnym. W rzeczywistej logistyce jest całkowicie bezużyteczna. Służy jako punkt odniesienia udowadniający, że inne zaimplementowane algorytmy faktycznie wykonują inteligentną optymalizację.
- **Heurystyki (NN, RNN):** Nadają się do bardzo dużych instancji, gdzie liczy się czas odpowiedzi, a nie idealne optimum. Algorytm RNN jest doskonałym kompromisem - pozwala uzyskać wynik bliski optymalnemu w akceptowalnym czasie. Nie nadaje się do zastosowań krytycznych, gdzie koszt trasy musi być absolutnie minimalny.
- **Branch and Bound (LC + UB):** Jest to najbardziej zaawansowana z badanych metod dokładnych. Nadaje się do problemów o średniej wielkości. Dzięki mechanizmowi redukcji macierzy i inteligentnemu przeszukiwaniu, potrafi znaleźć i udowodnić optimum znacznie szybciej niż Brute-Force.
- **Branch and Bound (DFS):** Jest to jedyna metoda dokładna, która nadaje się do systemów o bardzo ograniczonej pamięci operacyjnej, dzięki swojej liniowej złożoności pamięciowej $O(N)$.

Podsumowując: Wybór algorytmu zależy od priorytetu: jeśli kluczowa jest **gwarancja optimum**, wybieramy **B&B LC + UB**. Jeśli kluczowa jest **szybkość przy dużym N**, jedynym wyborem są **heurystyki (RNN)**. Metody dokładne przestają być efektywne, gdy rozmiar problemu uniemożliwia zakończenie obliczeń w racjonalnym czasie.

V. PODSUMOWANIE

Z punktu widzenia praktycznego, najlepszym kompromisem jest połączenie **metod LC (Lowest-Cost) z początkowym ograniczeniem UB z algorytmu RNN**. Taka konfiguracja najszybciej zbiega do wyniku optymalnego i potrafi udowodnić jego poprawność. W sytuacjach ekstremalnie ograniczonych zasobów pamięci RAM (np. systemy wbudowane), należy jednak skłonić się ku metod **DFS**, która mimo dłuższego czasu pracy, gwarantuje stałą, niską zajętość pamięci. Ostatecznie metoda Podziału i Ograniczeń stanowi potężne narzędzie optymalizacyjne, jednak jej zasięg ograniczony jest do małych i średnich instancji. Dla problemów rzeczywistych o rozmiarach $N > 30$ niezbędne staje się wykorzystanie metod metaheurystycznych.

LITERATURA

- [1] *VLSI TSP Data*, University of Waterloo. [Online]. Dostępne: <https://www.math.uwaterloo.ca/tsp/vlsi/index.html>
- [2] Mecalux, *Problem komiwojazera w logistyce*. [Online]. Dostępne: <https://www.mecalux.pl/blog/problem-komiwojazera>
- [3] J. Rudy, *Złożoność obliczeniowa i algorytmy - Materiały pomocnicze*, PWr. [Online]. Dostępne: <https://kam.pwr.edu.pl/jaroslav-rudypwr-edu-pl/files/zto/lab4.pdf>
- [4] J. Nowosad, *Branch and Bound - Wykłady*. [Online]. Dostępne: <https://jakubnowosad.com/ahod/07-branch-and-bound.html#1>
- [5] W. Cook et al., *The Traveling Salesman Problem*, University of Waterloo. [Online]. Dostępne: <https://www.math.uwaterloo.ca/tsp/index.html>